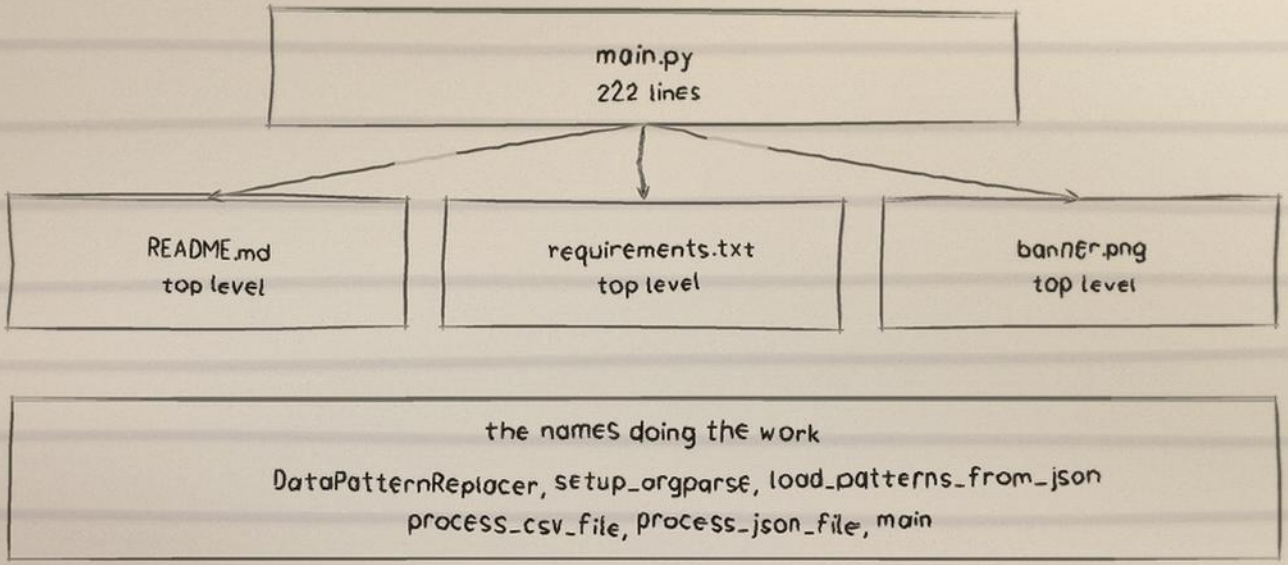
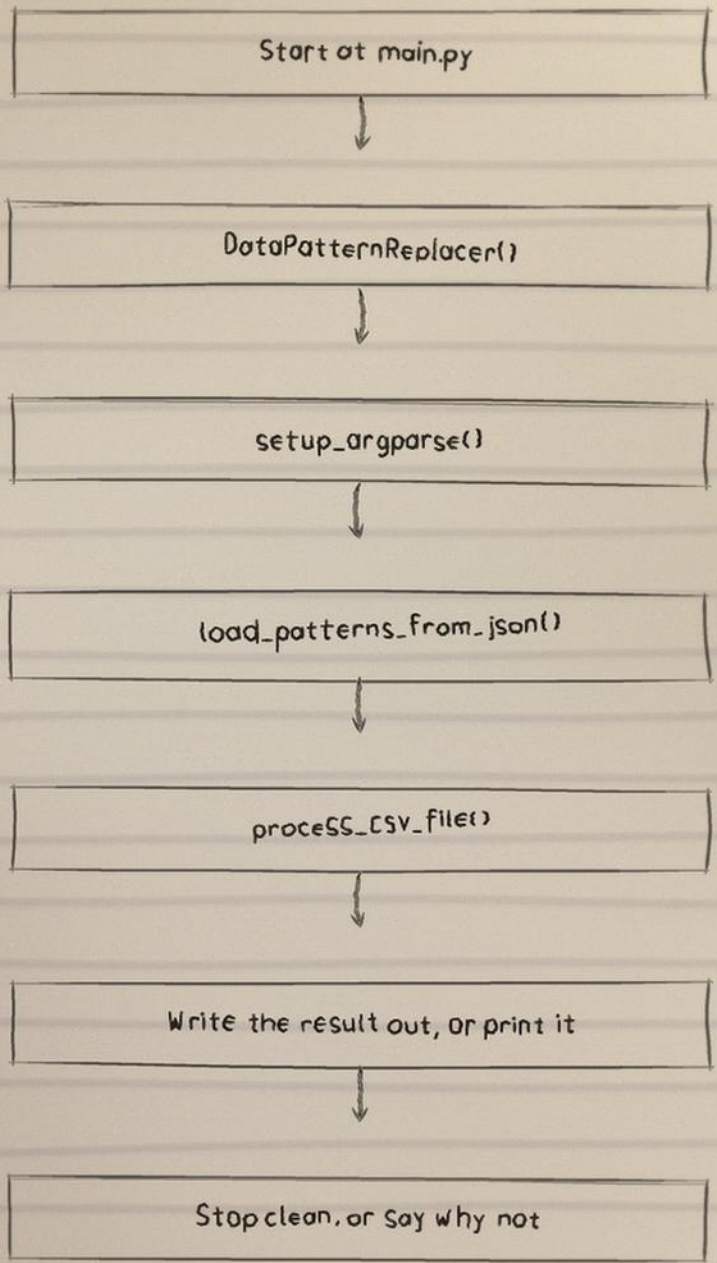
Datascan Data Pattern Replacer: how it is put together



Note: --inputformat and --outputformat must match. The tool reads and writes in the same format and does not convert between text, JSON, and CSV; a mismatched pair is refused

sketched from the files

Datascan Data Pattern Replacer: the run, step by ste



If a step fails, the honest run says so and stops.

traced, not guessed, GK

Product map: what Datascan Data Pattern Replacer is for

WHAT IT IS

Note: --inputformat and --outputformat must match. The tool reads and writes in the same format and does not convert between text, JSON, and CSV; a mismatched pair is refused with.

WHAT GOES IN

The files at the top of the tree: main.py, README.md, requirements.txt.

WHAT COMES OUT

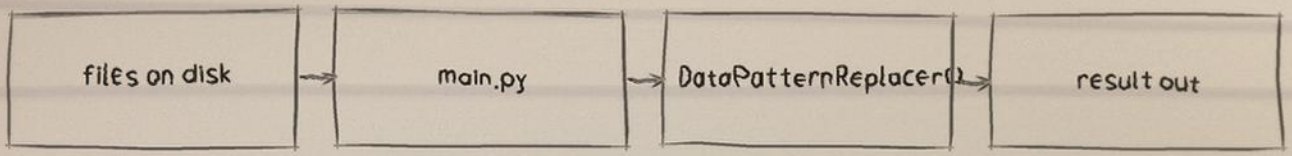
What the top files produce when they run or get read. Nothing else is promised.

WHERE IT STOPS

No service behind it, no account, no upload. If a path is untested, VISUAL.md says so.

mapped by hand, GK

Datascan Data Pattern Replacer: data flow, in one direction

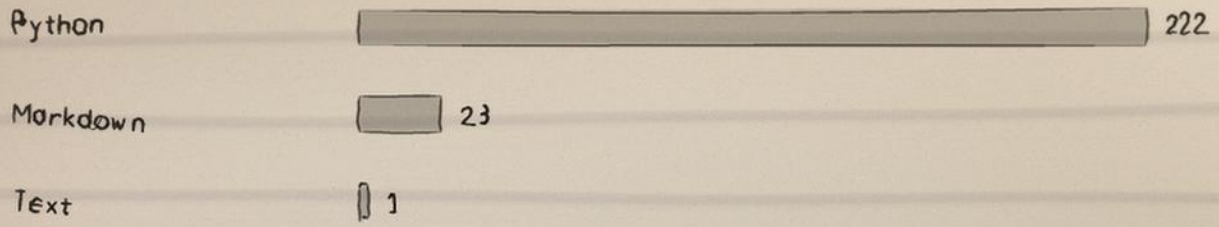


One direction. Anything the code does not do is not drawn.

second sheet, GK

Datascan Data Pattern Replacer in numbers, counted 2026-10-30

Counted from the local snapshot of this repo. Method for every number: VISUAL.md.



5 files in the tree.
7 commits, first 2025-07-03, last 2026-10-30, from the local git log.
No test files by name in this snapshot.

counted and drawn, GK

Datascan Data Pattern Replacer: schematic of the apparatus

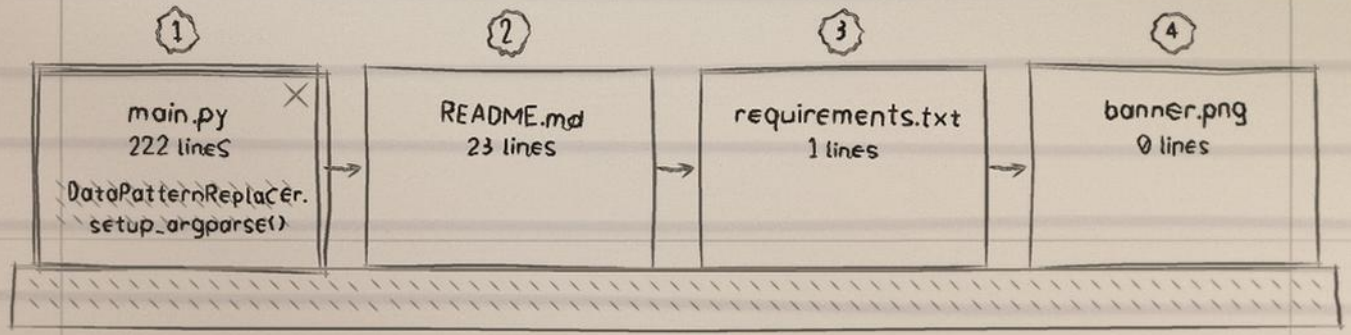
the system as a machine: parts, flow, and measurements

construction lines left in, as drawn

parts list

1. main.py
2. README.md
3. requirements.txt
4. banner.png

in flow order, first to last



base plate: the repo itself, sectioned

span: 5 files, 246 lines counted

turns of the crank so far: 7 commits, 2025-07-03 to 2026-10-30

measured and sketched, GK